Updated Test Plan

Order Management API

1. Introduction

This document covers the testing strategy for the Order Management API, a RESTful API built with Node.js and Express.js. The API handles customer orders and supports creating, reading, updating, and deleting orders through 5 endpoints. Data is stored using SQLite, which keeps records between server restarts. The goal of this test plan is to verify that each endpoint works correctly, handles bad input properly, and returns the right HTTP status codes and response data.

2. Testing Scope

The following endpoints are covered in this test plan:

- POST /orders - Create a new order
- GET /orders - Retrieve all orders
- GET /orders/:id - Retrieve a specific order by ID
- PATCH /orders/:id - Update the status of an order
- DELETE /orders/:id - Delete an order

Testing also covers error handling for missing fields, invalid status values, and requests made with IDs that do not exist in the database.

3. Testing Approach

Three types of testing are used in this plan:

Unit Testing

Unit tests check that individual endpoint functions handle input correctly on their own, without relying on external dependencies like a live database. The focus is on input validation, making sure the API returns the right error responses when required fields are missing or when an invalid value is passed in.

Integration Testing

Integration tests verify that the API endpoints work correctly when connected to the database. These tests cover the full CRUD lifecycle, creating an order, retrieving it, updating its status, deleting it, and then confirming it is no longer accessible. An in-memory SQLite database is used so tests run cleanly without touching the real data file.

Postman Testing

Postman tests validate the API from an end-user perspective by sending real HTTP requests and checking the responses. Each request has test scripts that verify the status code, response body structure, required fields, and response time. These tests were run against the locally hosted server on http://localhost:3000.

4. Test Environment

- Operating System: macOS
- Runtime: Node.js v20
- Framework: Express.js
- Database: SQLite (file-based via better-sqlite3, in-memory for automated tests)
- Testing Tools: Jest v29, Supertest v6, Postman
- Server URL: http://localhost:3000

5. Test Deliverables

- This test plan document
- Jest unit and integration test file (app.test.js)
- Postman collection with test scripts added to each request
- Screenshots of Postman test results
- Jest terminal output showing all tests passing

6. Unit Test Cases

The following table documents the unit test cases designed to verify input validation and error handling at the endpoint level.

Test Case ID	Description	Input	Expected Output	Pass/Fail Criteria
UT-001	POST /orders - missing customerName	{ item: "Laptop", quantity: 1 }	400 Bad Request with error message	Passes if status is 400 and error message is returned
UT-002	POST /orders - missing item	{ customerName: "Jane", quantity: 1 }	400 Bad Request with error message	Passes if status is 400 and error message is returned
UT-003	POST /orders - missing quantity	{ customerName: "Jane", item: "Laptop" }	400 Bad Request with error message	Passes if status is 400 and error message is returned
UT-004	PATCH /orders/:id - invalid status value	{ status: "InvalidStatus" }	400 Bad Request	Passes if status is 400

UT-005	GET /orders/:id - non-existent ID	/orders/non-existent-id	404 Not Found with error message	Passes if status is 404 and error message is returned
UT-006	DELETE /orders/:id - non-existent ID	/orders/non-existent-id	404 Not Found	Passes if status is 404

7. Integration Test Cases

The following table documents the integration test cases designed to verify that the API endpoints interact correctly with the SQLite database across the full order lifecycle.

Test Case ID	Description	Input	Expected Output	Pass/Fail Criteria
IT-001	Create order with valid data	{ customerName: "Jane Doe", item: "Laptop", quantity: 1 }	201 Created with order object and transactionId	Passes if order is saved to DB and response contains id and transactionId
IT-002	Retrieve all orders	GET /orders	200 OK with array of orders	Passes if response is an array with at least 1 order
IT-003	Retrieve a specific order by ID	GET /orders/:id	200 OK with correct order object	Passes if correct order is returned with matching ID
IT-004	Update order status	{ status: "Shipped" }	200 OK with updated order	Passes if status field is updated correctly in the database
IT-005	Delete an order	DELETE /orders/:id	200 OK with success message	Passes if order is removed from the database
IT-006	Retrieve deleted order	GET /orders/:id after delete	404 Not Found	Passes if deleted order is no longer accessible

8. Postman Test Cases

The following table documents all Postman test scripts added to each request in the collection. Tests were run against the local server and validate status codes, response body structure, headers, and response times.

Request	Test	Validation	Expected Result
POST /orders	Status code check	pm.response.to.have.status(201)	201 Created
POST /orders	Order object exists	jsonData.order.id exists	ID is present in response

POST /orders	TransactionId exists	jsonData.transactionId exists	TransactionId present from payment gateway
POST /orders	Content-Type is JSON	pm.response.to.have.header(Content-Type)	Header is present
POST /orders	Response time check	responseTime below 5000ms	Under 5 seconds
GET /orders	Status code check	pm.response.to.have.status(200)	200 OK
GET /orders	Response is array	jsonData is an array	Array returned
GET /orders	Content-Type is JSON	pm.response.to.have.header(Content-Type)	Header is present
GET /orders	Response time check	responseTime below 3000ms	Under 3 seconds
GET /orders/:id	Status code check	pm.response.to.have.status(200)	200 OK
GET /orders/:id	Required fields present	id, customerName, status, createdAt all exist	All fields present
GET /orders/:id	Response time check	responseTime below 3000ms	Under 3 seconds
PATCH /orders/:id	Status code check	pm.response.to.have.status(200)	200 OK
PATCH /orders/:id	Status updated correctly	jsonData.order.status equals "Shipped"	Status is Shipped
PATCH /orders/:id	Response time check	responseTime below 3000ms	Under 3 seconds
DELETE /orders/:id	Status code check	pm.response.to.have.status(200)	200 OK
DELETE /orders/:id	Deletion confirmed	jsonData.message equals "Order deleted successfully."	Message confirmed
DELETE /orders/:id	Response time check	responseTime below 3000ms	Under 3 seconds
GET /orders/fake-id-999	Status code check	pm.response.to.have.status(404)	404 Not Found
GET /orders/fake-id-999	Error message returned	jsonData.error equals "Order not found."	Error message present
GET /orders/fake-id-999	Response time check	responseTime below 3000ms	Under 3 seconds

9. Postman Test Results

All Postman tests were executed against the running local server. The table below summarizes the results. Screenshots of each individual test run showing the green PASSED indicators are included after the table.

Request	Tests Run	Passed	Failed	Status Code
POST /orders (Create Order)	Status code, order object, transactionId, Content-Type, response time	5	0	201 Created
GET /orders (Get All Orders)	Status code, array check, Content-Type, response time	4	0	200 OK
GET /orders/:id (Get Order By ID)	Status code, fields check, response time	3	0	200 OK
PATCH /orders/:id (Update Status)	Status code, status value, response time	3	0	200 OK
DELETE /orders/:id (Delete Order)	Status code, deletion message, response time	3	0	200 OK
GET /orders/fake-id-999 (Error Case)	Status code, error message, response time	3	0	404 Not Found
TOTAL		**21**	**0**	**All Passed**

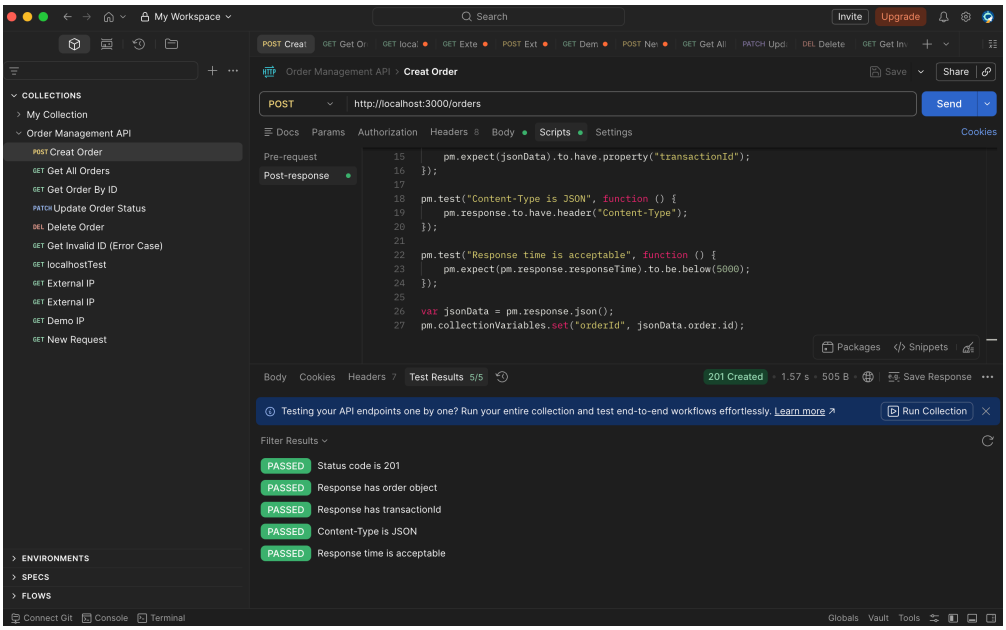

Figure 1: POST Create Order - Test Results

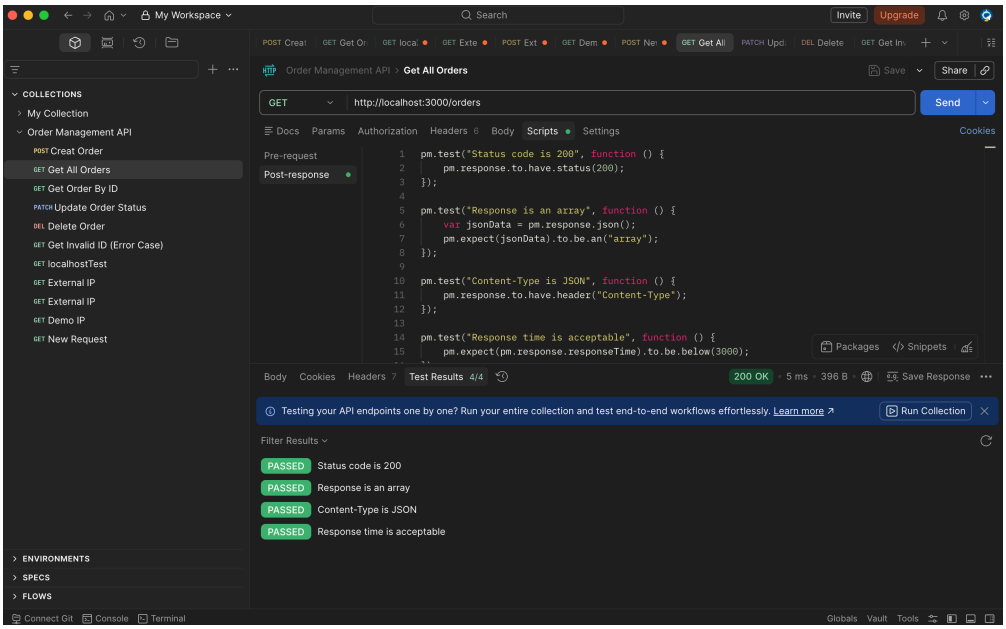

Figure 2: GET All Orders - Test Results

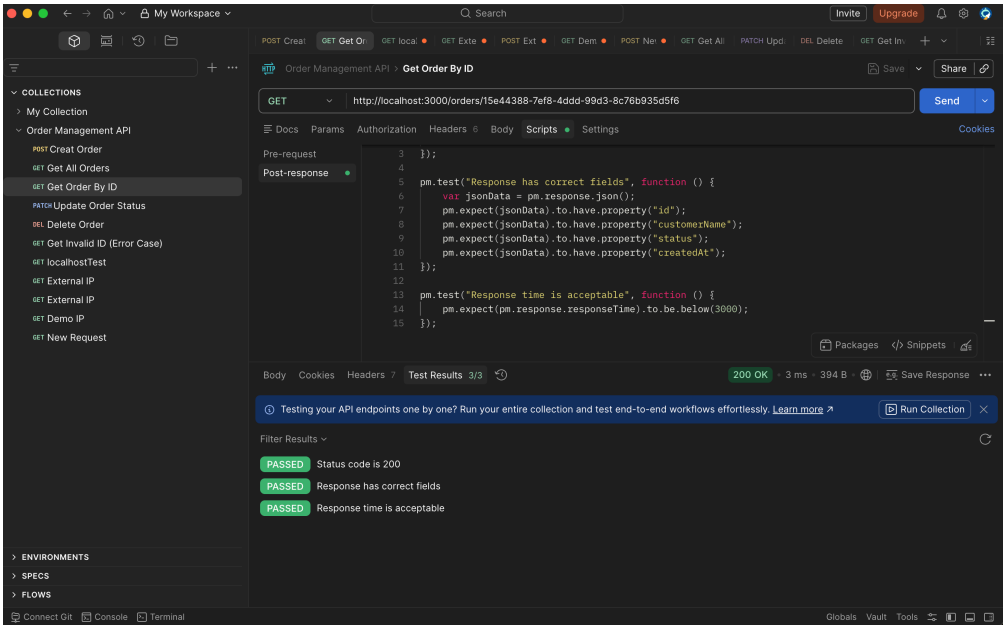

Figure 3:GET Order By ID - Test Results

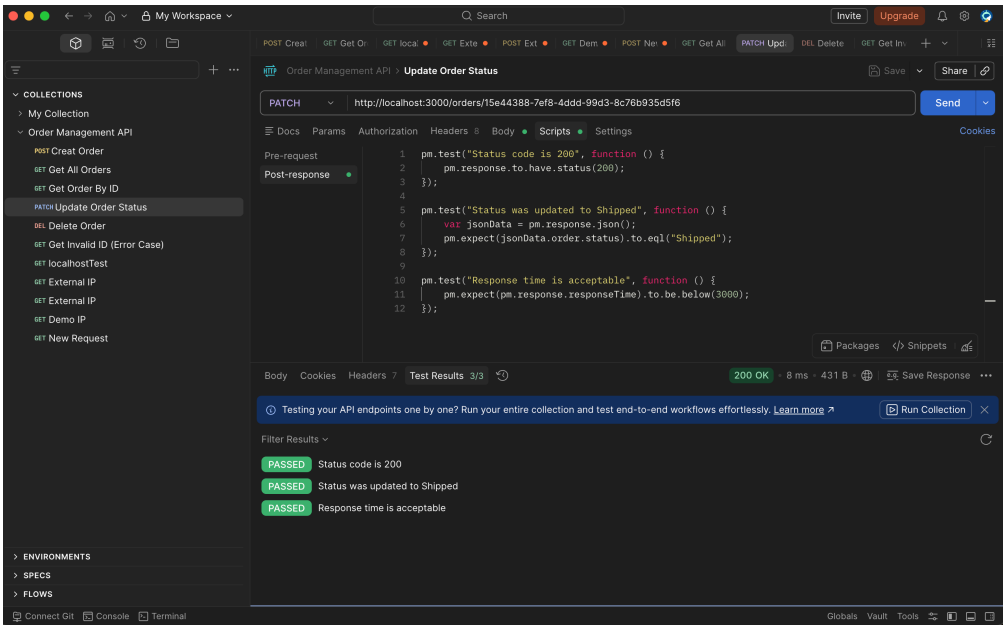

Figure 4: PATCH Update Order Status - Test Results

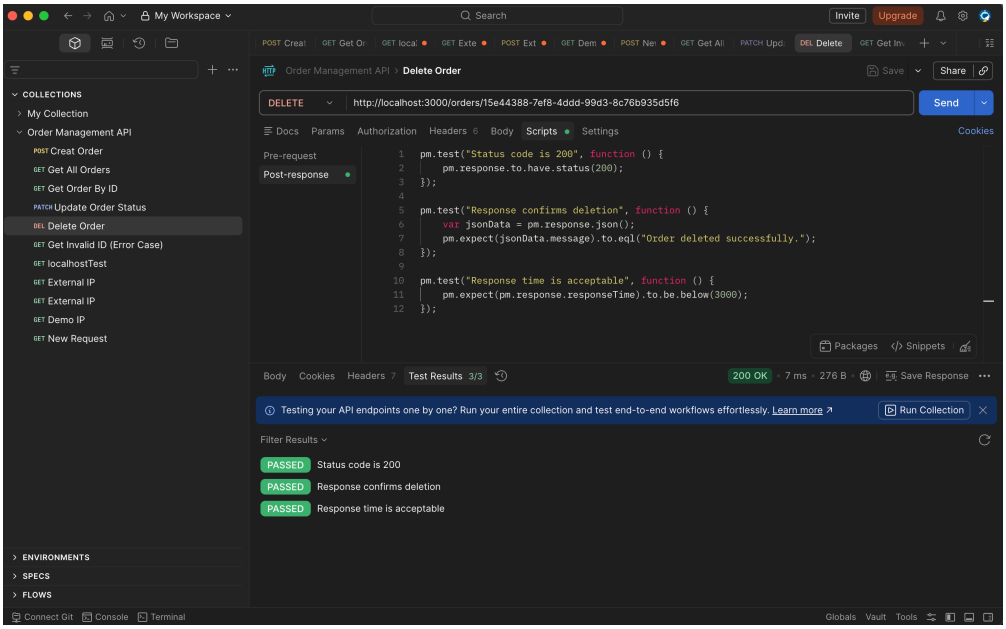

Figure 5: DELETE Order - Test Results

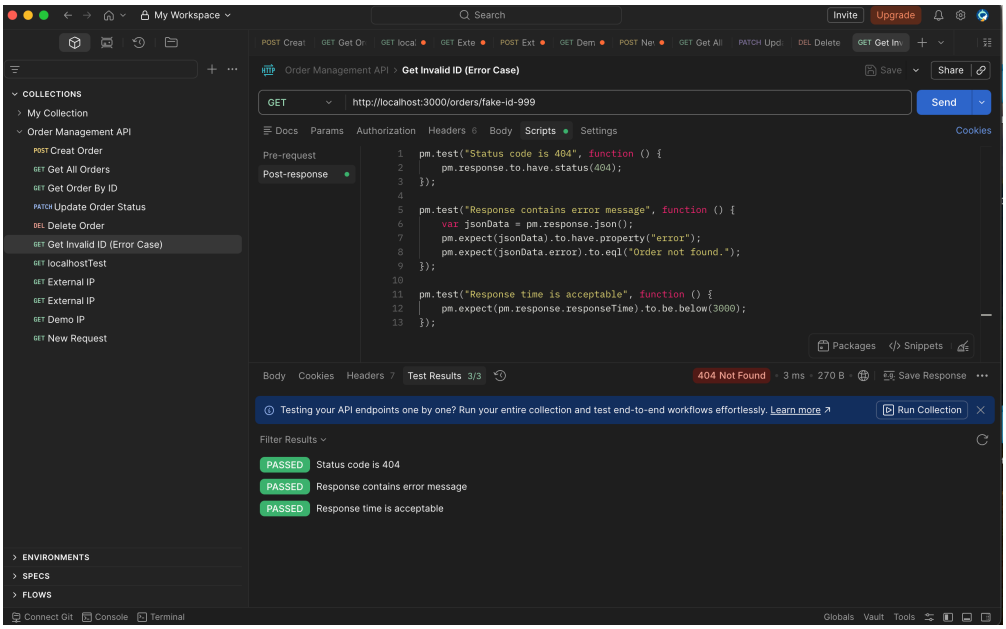

Figure 6: GET Invalid ID - Test Results

10. Jest Test Results (Module 11)

All automated unit and integration tests were run using Jest and Supertest. The table below documents the results of each test case. All 12 tests passed with a total run time of under 1 second.

Type	Test Name	Result	Duration
Unit	POST /orders - missing customerName returns 400	PASSED	15ms
Unit	POST /orders - missing item returns 400	PASSED	4ms
Unit	POST /orders - missing quantity returns 400	PASSED	1ms
Unit	PATCH /orders/:id - invalid status returns 400	PASSED	1ms
Unit	GET /orders/:id - non-existent ID returns 404	PASSED	1ms
Unit	DELETE /orders/:id - non-existent ID returns 404	PASSED	1ms
Integration	POST /orders - creates order successfully	PASSED	103ms
Integration	GET /orders - returns array of orders	PASSED	1ms
Integration	GET /orders/:id - retrieves specific order	PASSED	1ms
Integration	PATCH /orders/:id - updates order status	PASSED	1ms
Integration	DELETE /orders/:id - deletes order	PASSED	1ms
Integration	GET /orders/:id - deleted order returns 404	PASSED	1ms

TOTAL	**12 Tests**	**12 Passed**	**0 Failed**

Figure 7: Terminal Output - npm test

11. Test Summary (Module 11)

All tests across all three testing phases passed successfully. No failures or unexpected behaviors were encountered during testing.

- Unit Tests: 6 out of 6 passed
- Integration Tests: 6 out of 6 passed
- Postman Tests: 21 out of 21 passed

The Order Management API correctly handles valid requests, returns appropriate error responses for invalid or missing input, and interacts with the database as expected across all CRUD operations.

Module 12: Unit Testing Update

12. Module 12 Testing Objectives (SMART)

The following objectives were defined for this unit testing phase, written to be Specific, Measurable, Achievable, Relevant, and Time-bound:

- Specific: Write unit tests for all 5 controller functions (createOrder, getAllOrders, getOrderById, updateOrderStatus, deleteOrder), covering valid inputs, invalid inputs, boundary conditions, edge cases, and mocked database errors for each function.
- Measurable: Achieve 100% statement, branch, function, and line coverage on the orderController.js file as measured by Jest's built-in coverage reporter.
- Achievable: Use Jest's jest.mock() and jest.fn() capabilities to mock the orderService module, allowing each controller function to be tested in complete isolation from the database.
- Relevant: Controller-level unit testing directly validates the business logic and input handling that determines what HTTP responses users receive, which is the core functionality of the API.
- Time-bound: All unit tests were written, executed, and verified within the Module 12 assignment period, with results documented in this updated test plan.

13. Jest Setup and Configuration

Jest was configured using a jest.config.js file in the project root with the following settings:

- testEnvironment: node - runs tests in a Node.js environment rather than a browser environment
- testMatch: ['**/tests/**/*.test.js', '**/*.test.js'] - picks up test files from both the tests/ directory and the root
- collectCoverageFrom: ['controllers/**/*.js', 'services/**/*.js'] - measures coverage only on the controller and service files
- coverageReporters: ['text', 'lcov'] - outputs coverage as a terminal table and an lcov file

The package.json scripts section was updated to include a separate coverage command: npm run coverage runs jest --coverage to generate the full coverage report.

14. Mocking Strategy

The entire orderService module was mocked using jest.mock('../services/orderService') at the top of the test file. This completely replaces the real service with Jest mock functions, meaning no actual database calls are made during any of the unit tests. Each individual test then uses mockResolvedValue() to control what the service returns, or mockRejectedValue() to simulate a database error being thrown.

Mock request and response objects were created using a helper function called mockRes() which returns an object with status and json as Jest spy functions. This allows the tests to assert exactly which status code and response body the controller sent back without needing a running HTTP server.

After each test, jest.clearAllMocks() is called in the afterEach block to reset all mock call counts and return values, keeping each test fully independent of the others.

15. Module 12 Unit Test Cases

The following table documents all new unit test cases added in Module 12. These tests expand on the Module 11 unit tests by adding valid input testing, boundary condition testing, edge case testing, and mocked database error testing for every controller function.

Test Case ID	Description	Input	Expected Output	Pass/Fail Criteria
UT-007	createOrder - valid input returns 201	{ customerName: "Jane Doe", item: "Laptop", quantity: 1 }	201 Created with order object and transactionId	Passes if service is called with correct args and 201 is returned
UT-008	createOrder - quantity of 1 (minimum boundary)	{ customerName: "John", item: "Mouse", quantity: 1 }	201 Created	Passes if minimum valid quantity of 1 is accepted
UT-009	createOrder - large quantity (upper boundary)	{ customerName: "Bulk Corp", item: "Keyboard", quantity: 1000 }	201 Created	Passes if large quantity value is accepted
UT-010	createOrder - quantity of zero (edge case)	{ customerName: "Jane", item: "Laptop", quantity: 0 }	400 Bad Request	Passes if zero quantity is rejected with correct error message
UT-011	createOrder - negative quantity (edge case)	{ customerName: "Jane", item: "Laptop", quantity: -5 }	400 Bad Request	Passes if negative quantity is rejected with correct error message
UT-012	createOrder - database error (mocked)	Service throws: Error('Database connection failed.')	500 Internal Server Error	Passes if error is caught and 500 is returned with error message
UT-013	getAllOrders - returns list of orders	GET /orders	200 OK with array	Passes if service returns array and 200 is returned

UT-014	getAllOrders - empty database (edge case)	No orders exist in DB	200 OK with empty array	Passes if empty array is returned with 200
UT-015	getAllOrders - database error (mocked)	Service throws: Error('Database connection failed.')	500 Internal Server Error	Passes if 500 is returned
UT-016	getOrderById - valid ID returns correct order	GET /orders/abc-1	200 OK with matching order object	Passes if service is called with correct ID and order is returned
UT-017	getOrderById - non-existent ID returns 404	GET /orders/non-existent-id	404 Not Found	Passes if 404 is returned when service returns null
UT-018	getOrderById - database error (mocked)	Service throws: Error('Database connection failed.')	500 Internal Server Error	Passes if 500 is returned
UT-019	updateOrderStatus - valid status returns 200	{ status: "Shipped" }	200 OK with updated order	Passes if order is updated and 200 is returned
UT-020	updateOrderStatus - all 5 valid statuses (boundary)	Each of: Pending, Processing, Shipped, Delivered, Cancelled	200 OK for each	Passes for each valid status value
UT-021	updateOrderStatus - invalid status value	{ status: "Flying" }	400 Bad Request	Passes if invalid status is rejected and service is not called
UT-022	updateOrderStatus - missing status (edge case)	{}	400 Bad Request	Passes if missing status is rejected
UT-023	updateOrderStatus - empty string status (edge case)	{ status: "" }	400 Bad Request	Passes if empty string is rejected
UT-024	updateOrderStatus - order not found returns 404	Non-existent order ID	404 Not Found	Passes if 404 is returned and update is not called
UT-025	updateOrderStatus - database error (mocked)	Service throws: Error('Database connection failed.')	500 Internal Server Error	Passes if 500 is returned
UT-026	deleteOrder - valid ID returns 200	DELETE /orders/abc-1	200 OK with success message	Passes if order is deleted and correct message is returned

UT-027	deleteOrder - non-existent ID returns 404	DELETE /orders/non-existent	404 Not Found	Passes if 404 is returned and delete is not called
UT-028	deleteOrder - database error (mocked)	Service throws: Error('Database connection failed.')	500 Internal Server Error	Passes if 500 is returned

16. Assertion Methods Used

The following Jest assertion methods were used across the test suite:

- expect(res.status).toHaveBeenCalledWith(201) - verifies the controller called res.status() with the correct HTTP status code
- expect(res.json).toHaveBeenCalledWith(expect.objectContaining({...}))- verifies the response body contains specific fields without requiring an exact match
- expect(res.json).toHaveBeenCalledWith({ error: '...' }) - verifies the exact error message returned for validation failures
- expect(orderService.createOrder).toHaveBeenCalledWith({...}) - verifies the service was called with the correct arguments
- expect(orderService.createOrder).not.toHaveBeenCalled() - verifies the service was NOT called when input validation fails
- expect(orderService.deleteOrder).toHaveBeenCalledWith('abc-1') - verifies the correct order ID was passed to the service

17. Module 12 Jest Test Results

All 37 tests across both test suites passed successfully. The table below documents every test case from both test files along with its result and duration.

Suite	Test Name	Result	Duration
app.test.js	POST /orders - missing customerName returns 400	PASSED	15ms
app.test.js	POST /orders - missing item returns 400	PASSED	4ms
app.test.js	POST /orders - missing quantity returns 400	PASSED	1ms
app.test.js	PATCH /orders/:id - invalid status returns 400	PASSED	1ms
app.test.js	GET /orders/:id - non-existent ID returns 404	PASSED	1ms
app.test.js	DELETE /orders/:id - non-existent ID returns 404	PASSED	1ms
app.test.js	POST /orders - creates order successfully	PASSED	103ms
app.test.js	GET /orders - returns array of orders	PASSED	1ms

app.test.js	GET /orders/:id - retrieves specific order	PASSED	1ms
app.test.js	PATCH /orders/:id - updates order status	PASSED	1ms
app.test.js	DELETE /orders/:id - deletes order	PASSED	1ms
app.test.js	GET /orders/:id - deleted order returns 404	PASSED	1ms
orderController.test.js	createOrder - should create an order and return 201 with valid input	PASSED	5ms
orderController.test.js	createOrder - should create an order with quantity of 1 (boundary condition)	PASSED	1ms
orderController.test.js	createOrder - should create an order with a large quantity (boundary condition)	PASSED	1ms
orderController.test.js	createOrder - should return 400 when customerName is missing	PASSED	1ms
orderController.test.js	createOrder - should return 400 when item is missing	PASSED	1ms
orderController.test.js	createOrder - should return 400 when quantity is missing	PASSED	1ms
orderController.test.js	createOrder - should return 400 when quantity is zero (edge case)	PASSED	1ms
orderController.test.js	createOrder - should return 400 when quantity is negative (edge case)	PASSED	1ms
orderController.test.js	createOrder - should return 500 when service throws a database error	PASSED	1ms
orderController.test.js	getAllOrders - should return 200 with an array of orders	PASSED	1ms
orderController.test.js	getAllOrders - should return 200 with empty array when no orders exist (edge case)	PASSED	1ms
orderController.test.js	getAllOrders - should return 500 when service throws a database error	PASSED	1ms
orderController.test.js	getOrderById - should return 200 with correct order when valid ID is provided	PASSED	1ms
orderController.test.js	getOrderById - should return 404 when order ID does not exist	PASSED	1ms
orderController.test.js	getOrderById - should return 500 when service throws a database error	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 200 and update status with valid status value	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 200 for each valid status value (boundary conditions)	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 400 when status is invalid	PASSED	1ms

orderController.test.js	updateOrderStatus - should return 400 when status is missing (edge case)	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 400 when status is empty string (edge case)	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 404 when order does not exist	PASSED	1ms
orderController.test.js	updateOrderStatus - should return 500 when service throws a database error	PASSED	1ms
orderController.test.js	deleteOrder - should return 200 and delete order when valid ID is provided	PASSED	1ms
orderController.test.js	deleteOrder - should return 404 when order does not exist	PASSED	1ms
orderController.test.js	deleteOrder - should return 500 when service throws a database error	PASSED	1ms
TOTAL	**37 Tests across 2 Suites**	**37 Passed**	**0 Failed**

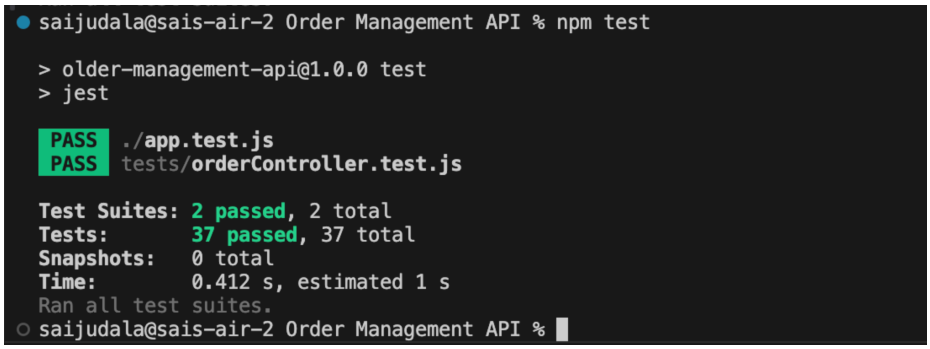

Figure 8: npm Test Terminal

18. Code Coverage Report

The coverage report was generated by running npm run coverage. The table below shows the coverage results for each file. The orderController.js file achieved 100% across all metrics, meaning every statement, branch, function, and line was exercised by the unit tests. The orderService.js file shows lower coverage because it is mocked in the unit tests and its internal database logic is not directly executed during controller-level testing.

File	% Statements	% Branch	% Functions	% Lines
controllers/orderController.js	100	100	100	100
services/orderService.js	41.66	0	0	45.45
All files	**79.41**	**83.33**	**38.46**	**80.95**

```
saijudala@sais-air-2 Order Management API % npm run coverage

> older-management-api@1.0.0 coverage
> jest --coverage

 PASS  tests/orderController.test.js
 PASS  ./app.test.js
----------------------|---------|----------|---------|---------|-------------------------
File                  | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------|---------|----------|---------|---------|-------------------------
All files             |   79.41 |    83.33 |   38.46 |   80.95 |
 controllers          |     100 |      100 |     100 |     100 |
  orderController.js  |     100 |      100 |     100 |     100 |
 services             |   41.66 |        0 |       0 |   45.45 |
  orderService.js     |   41.66 |        0 |       0 |   45.45 | 17-18,23-35,39,43,47-48,52
----------------------|---------|----------|---------|---------|-------------------------

Test Suites: 2 passed, 2 total
Tests:       37 passed, 37 total
Snapshots:   0 total
Time:        0.448 s, estimated 1 s
Ran all test suites.
saijudala@sais-air-2 Order Management API %
```

Figure 9: npm Run Coverage

19. Updated Test Summary

All tests across all testing phases passed successfully. The Module 12 unit testing phase added 25 new controller-level unit tests using mocking, bringing the total test count to 37. No failures or unexpected behaviors were encountered.

- Module 11 Unit Tests (app.test.js): 6 out of 6 passed
- Module 11 Integration Tests (app.test.js): 6 out of 6 passed
- Module 11 Postman Tests: 21 out of 21 passed
- Module 12 Controller Unit Tests (orderController.test.js): 25 out of 25 passed
- Total Jest Tests: 37 out of 37 passed across 2 test suites
- orderController.js Coverage: 100% statements, 100% branches, 100% functions, 100% lines

The restructured codebase separates concerns into controllers and services, making the application more testable and maintainable. The mocking approach used in Module 12 ensures each controller function is tested in complete isolation, which is the correct approach for unit testing.